

Advanced Statistics in

Bernd Figner
bernd.figner@ru.nl

Week 2: Sept 14, 2026

1

1

Before we start with the new stuff...

- **Can you hear me OK?**
- **Homework**
 - A) too little, B) too much, C) just right?
 - A) too easy, B) too difficult, C) just right?
 - Uploaded to Pitch2Peer? Questions/comments?
 - Example Solutions (R script): 13:00 on Brightspace Week 1
→ Homework → Example Solution
 - Peergrading during lab session this afternoon (deadline Sept 21 before lecture)
- **Discussion Boards: current $n < 10$; goal $n > 140$**
- **Questions/suggestions/comments/...?**

2

Overview

- **Book chapters 1 and 2: important concepts**
 - Population vs. sample; descriptive vs. inference statistics
 - Types of data and probability distributions (discrete vs. continuous)
 - Central limit theorem; central tendency, variability, shape dispersion
- **More R basics: data management**
 - Recap/leftovers/add-on from last week
 - Saving and loading data and your work
 - Long vs. wide format: reshaping
 - Selecting parts of a data frame (subset)
- **Homework**

3

3

Chapter 1

- Samples and populations
 - Sample statistics versus population parameters
- Descriptive versus inference statistics
 - Descriptive statistics $\leftrightarrow$ exploratory analysis
 - Inference statistics $\leftrightarrow$ confirmatory analysis
- Data: discrete versus continuous
- Measures to describe the central tendency in data
- Measures to describe the dispersion in data

4

4

Samples and populations

- Population = abstract construct (often assumed to be infinitely large)
- In reality, we typically deal with samples in research
- BUT: We are typically interested in conclusions about populations → We want to generalize from sample to population

Examples

- We assess working memory performance in a sample of 100 sleep-deprived and 100 well-rested students
→ We want to draw conclusions about ALL sleep-deprived vs. well-rested young adults (or even all humans), not just about the 200 students in study
- We collect data from each 200 participants in 10 different Western countries on their (a) socio-economic status and (b) access to healthcare: We want to generalize from these 2000 individuals (and from these 10 countries) to the populations of these countries (and perhaps to ALL countries in the world)

5

5

Inference vs. descriptive stats

- We want to use the sample to make **inferences** about the population
 - Ex 1: Working memory performance in sleep-deprived *young adults* significantly worse than in well-rested *young adults*
 - Ex 2: Lower socio-economic status significantly associated with less access to healthcare in Western countries
 - We use the sample for conclusions about the population, but there is necessarily **uncertainty** (because we have data only from a sample, not the whole population); larger samples → less uncertainty
- Using a sample to make inferences about a population is the domain of **inference statistics**
- If we have an a priori hypothesis/research question and want to test this, this is often called **confirmatory** analysis

6

6

Inference vs. descriptive stats

- We also want to understand and **describe** our data (often done before confirmatory analysis)
- For example: How do the distributions of working memory performance in the two groups look like?
 - Central tendency: mean, median, mode
 - Dispersion: min, max, range, interquartile range, variance, SD
 - Shape: skew, kurtosis

→ This is the domain of **descriptive** statistics

If we do not have an a priori hypothesis/research question, but want to understand/describe the data...

... this is often called **exploratory** analysis

→ Distinction confirmatory vs exploratory is not always completely black and white

7

7

Chapter 2

- Common probability distributions for discrete and continuous data
 - Discrete: binomial, Poisson
 - Continuous: normal, lognormal, chi-square, t , F
- Skew, kurtosis
- Central limit theorem

8

8

Probability Distributions

- Conclusions about population based on sample are inherently uncertain
- Typical way to represent uncertainty is with **probabilities**
- To derive conclusions about population based on sample, necessary to assume specific probability distribution
→ How are the data distributed in the population?
- Good news: A small number of probability distributions capture many key characteristics of populations that are necessary for us to do statistics

9

9

Probability Distributions cont.

- Fundamental distinction: **discrete** or **continuous** distribution?
- **Discrete data:** restricted in the values that can occur, e.g.:
 - Binary data: 0/1; correct/incorrect; healthy/sick; left/right; ...
 - Frequency data (aka count data): How often does something occur (0, 1, 2, 3, ...)
- **Continuous data:** all possible intermediate values can occur (at least in principle)
 - How tall are the students in this class
 - Reaction times in a Stroop task (lower bound at 0!)
- Sometimes, not always that clear-cut: e.g., answers on Likert-type scales (e.g., values 1, 2, 3, 4, 5)
- [Other distinction: nominal, ordinal, interval, ratio]

10

10

Probability Distributions cont

How to summarize/describe probability distributions?

- **Central tendency** ('first moment'): mean, median, mode
- **Variability** ('second moment'): range, interquartile range, sum of squares, variance, standard deviation
- **Shape**: symmetric/asymmetric, unimodal, multimodal, uniform, ...
 - **Skew** ('third moment'): unimodal asymmetric; positive, 0, negative
 - **Kurtosis** ('fourth moment'): How much of the distribution is in the middle versus the tails; leptokurtic, mesokurtic, platykurtic

11

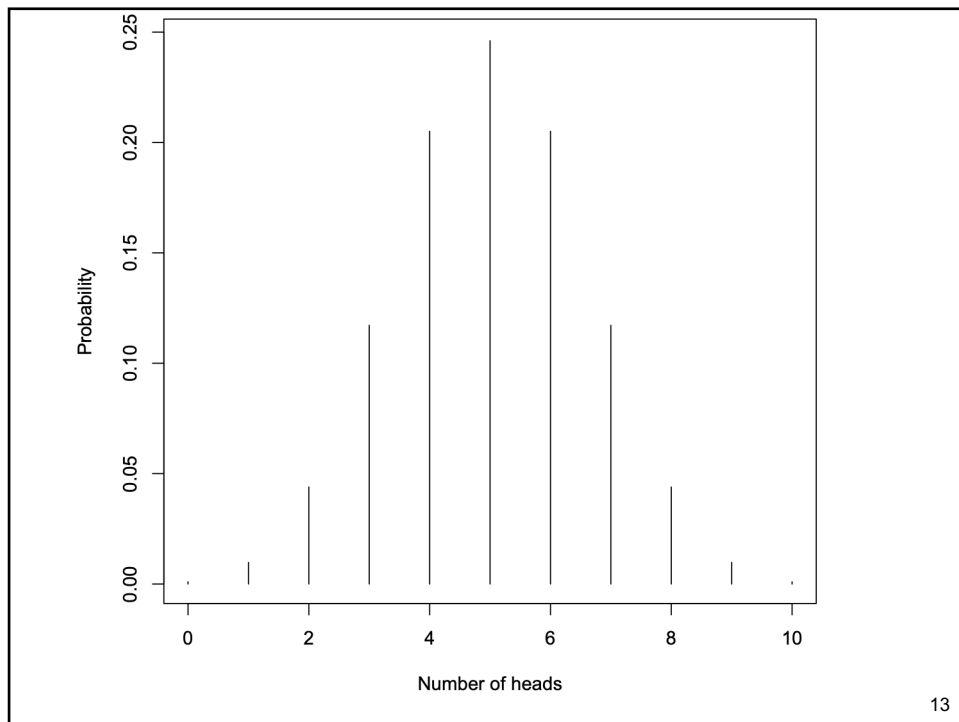
11

Probability **Mass** Function (pmf)

- **Discrete data**: What is the probability for each possible value
- **For example**: We flip a fair coin 10 times; what is the probability for each possible outcome (0 to 10 heads)?
- Since the different outcomes are mutually exclusive, the sum of all the probabilities has to be 1

12

12



13

R code (see Baguley book p. 67/68)

First, compute the probabilities

```
heads <- 0:10
prob_mass <- dbinom(heads, 10, .5)
prob_mass # to have a look at the numbers
0.0009765625 0.0097656250 0.0439453125 0.1171875000 0.2050781250
0.2460937500 0.2050781250 0.1171875000 0.0439453125 0.0097656250
0.0009765625
```

Second, create the figure

```
plot(heads, prob_mass, pch = NA, xlab = 'Number of
heads', ylab = 'Probability')
segments(x0 = heads, y0 = 0, x1 = heads, y1 = prob_mass)
```

or, simpler:

```
plot(heads, prob_mass, type = 'h', xlab = 'Number of
heads', ylab = 'Probability')
```

14

14

For continuous data?

- All intermediate values between upper and lower range are possible
- Thus: There are an infinite number of possible values
→ each specific value has a probability of 0
- Probabilities can be given for (arbitrarily small) ranges
- Probability **density** function (pdf)
- Examples: normal, F , t , chisquare, ...

Arguably the most important distribution in statistics is the **normal distribution** (aka **Gaussian distribution**)

15

15

Normal Distribution and CLT

- Unimodal, symmetrical, continuous, unbounded
- Described by mean and variance (or SD, aka sigma)
- The mean determines where the peak is
- The variance (SD) determines how wide or narrow the distribution is

Central Limit Theorem

- Many statistics (i.e., sample characteristics such as means and variances) follow a normal (or approximate normal) distribution under certain circumstances (e.g., large n)
- The **means** of the working memory performance of my 100 sleep-deprived and 100 well-rested students can be assumed to follow a **normal distribution** (even when the distribution of working memory performance itself is not normal)
- **Basis for many significance tests**

16

16

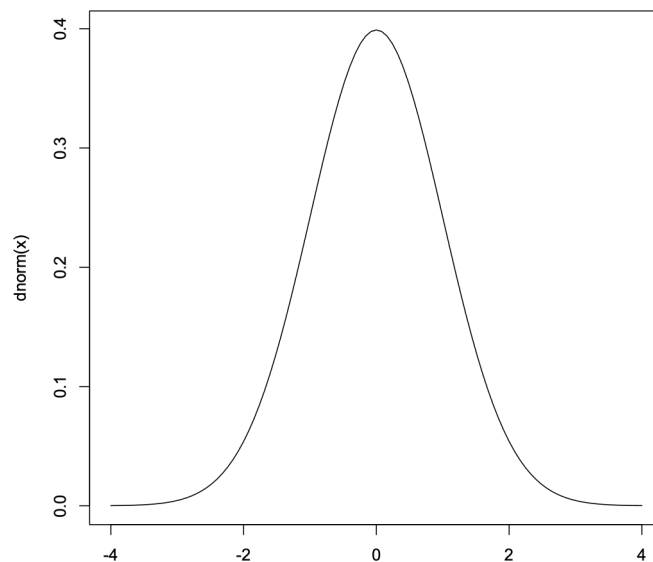
CLT cont.

- Let's say we draw samples many many many times from a population (e.g., sleep-deprived students WM performance)
 - We compute the mean of each sample
 - Under many circumstances, these means follow a normal distribution
 - This distribution is called the **sampling distribution** (in our example, the sampling distribution of the mean)
 - We describe this new distribution by its mean and *SD*
 - Its mean is expected to be the same as population mean
 - Its *SD* is called the standard error (aka standard error of the mean, *SEM*)
- How all of this is relevant for inference statistics will be explained soon in this course by Bill

17

17

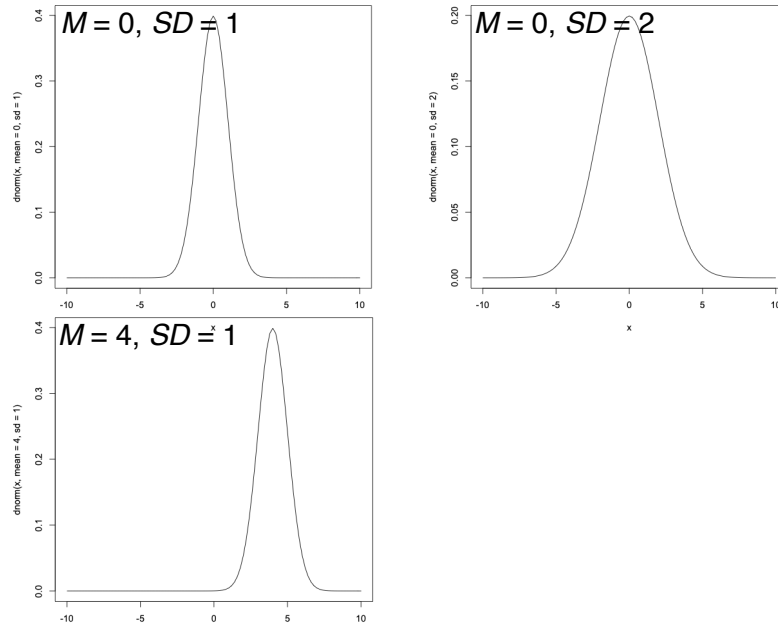
Probability density function with mean = 0 and *SD* = 1



R code: `curve(dnorm(x, mean = 0, sd = 1), xlim = c(-4, 4))`

18

Different means and SDs



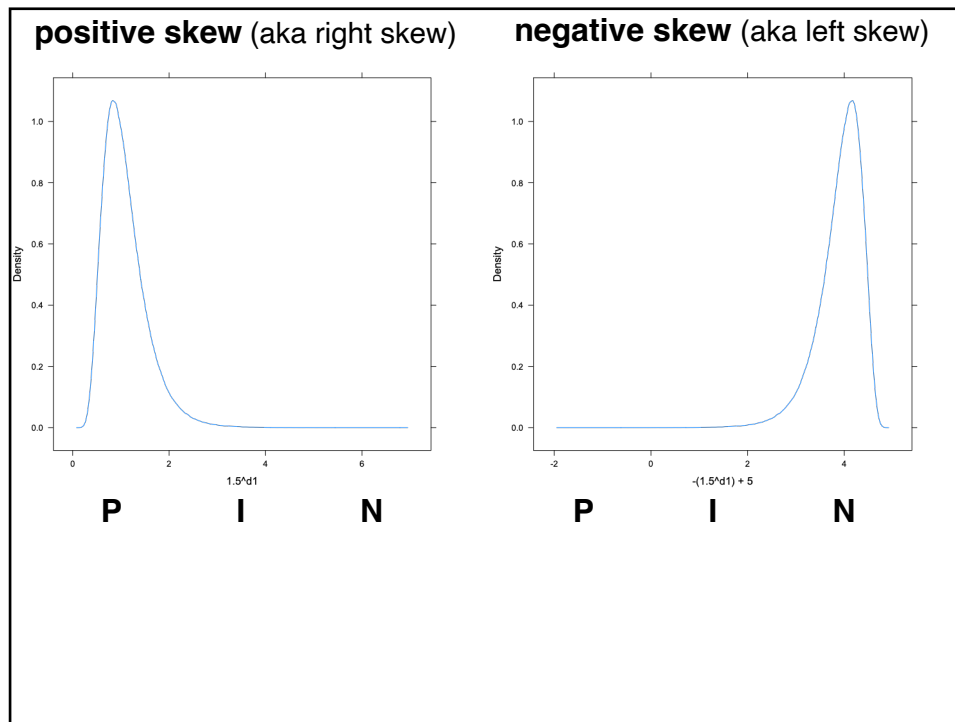
19

Skew and Kurtosis

- Skew and kurtosis describe aspects of the shape of a distribution
- A true normal distribution is not skewed and not kurtosed
- **Skew: Asymmetry of the distribution**
 - positive skew – no skew – negative skew

20

20



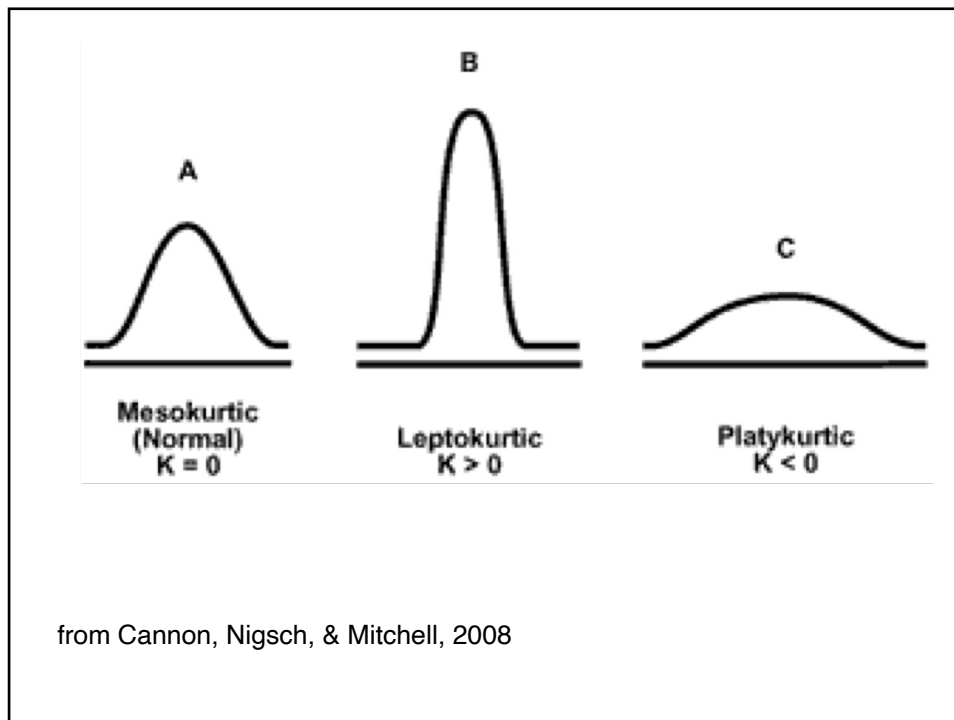
21

Skew and Kurtosis

- Skew and kurtosis describe aspects of the shape of a distribution
- A true normal distribution is not skewed and not kurtosed
- Skew: Asymmetry of the distribution
 - positive skew – no skew – negative skew
- **Kurtosis: Distribution of observations in the middle versus the tails**
 - leptokurtic -- platykurtic

22

22



23

Skew and Kurtosis

- Skew and kurtosis describe aspects of the shape of a distribution
- A true normal distribution is not skewed and not kurtosed
- Skew: Asymmetry of the distribution
 - positive skew – no skew – negative skew
- Kurtosis: Distribution of observations in the middle versus the tails
 - leptokurtic -- platykurtic
- **Why relevant: Sometimes, we apply transformations to turn skewed or kurtosed distributions into distributions that look more normal**

Caveat: transformations might create more problems than they purport to solve → try to avoid using transformations

24

24

Overview

- **Book chapters 1 and 2: important concepts**
 - Population vs. sample; descriptive vs. inference statistics
 - Types of data and probability distributions (discrete vs. continuous)
 - Central limit theorem; central tendency, variability, shape dispersion
- **More R basics: data management**
 - Recap/leftovers/add-on from last week
 - Saving and loading data and your work
 - Long vs. wide format: reshaping
 - Selecting parts of a data frame (subset)
- **Homework**

25

25

Add-on from last week

26

Important types of variables

- **Numeric**
 - Numbers (e.g., 7, 0, 120, 0.5, 21.376598)
→ `as.numeric()`, `is.numeric()`, `numeric()`
 - [there is also `as.integer()`, `is.integer()` `integer()`;
these are numbers without decimals]
- **Categorical**
 - **Unordered** categorical (e.g., "blue-eyed" and "brown-eyed")*
→ `as.factor()`, `is.factor()`, `factor()`
 - **Ordered** categorical (e.g., Age Group: "young" and "old")
→ `as.ordered()`, `is.ordered()`, `ordered()`
- **String**
 - Letters (e.g., "Teacher", "Student")*
→ `as.character()`, `is.character()`, `character()`
- **Date**
 - Dates (e.g., 21-06-1973, 06-21-73, 21-Jun-1973)

* In most (but not all!) cases " and ' are interchangeable

27

27

How to create a factor

Different variants to do that (roads to Rome...)

(1) Create numerical variable, then turn into factor

- 5 students and 5 lecturers → create a variable **job**
- Enter the data:

```
job <- c(1, 1, 1, 1, 1, 2, 2, 2, 2, 2)
```
- Then convert **job** to a factor:

```
f_job <- factor(job, levels = c(1:2), labels  
= c("Lecturer", "Student"))
```

28

28

(2) My preferred way: Use character entries from the beginning and use the command `as.factor()`

```
f_job <- as.factor(c("Lecturer", "Lecturer",
  "Lecturer", "Lecturer", "Lecturer", "Student",
  "Student", "Student", "Student", "Student"))
```

or, even simpler using `rep()`

```
f_job <- as.factor(c(rep("Lecturer", 5),
  rep("Student", 5)))
```

29

29

If you don't like scientific notation

- Scientific notation: stuff like **5.3e-6**
(same as 0.0000053)
- You can tell R to „penalize“ scientific notation:
`options(scipen = 5)`
- This tells R that, unless it saves 5 digits or more, NOT to use scientific notation
 - Higher positive numbers (such as `scipen = 10`)
→ biases output towards „normal“ number notation
 - Negative numbers (such as `scipen = -5`)
→ biases output towards scientific notation
- This affects ONLY how it's printed on the screen, NOT the actual calculations!
- To get that information, type `?options` and then search for `scipen` in the help window that opens

30

citation()

To cite R in publications use:

R Core Team (2024). *_R: A Language and Environment for Statistical Computing_*. R Foundation for Statistical Computing, Vienna, Austria. <<https://www.R-project.org/>>.

A BibTeX entry for LaTeX users is

```
@Manual{,
  title = {R: A Language and Environment for Statistical Computing},
  author = {{R Core Team}},
  organization = {R Foundation for Statistical Computing},
  address = {Vienna, Austria},
  year = {2024},
  url = {https://www.R-project.org/},
}
```

We have invested a lot of time and effort in creating R, please cite it when using it for data analysis. See also 'citation("pkgname")' for citing R packages.

Correct APA style:

R Core Team. (2024). *R: A language and environment for statistical computing*. (Version 4.4.2) [Computer software]. <https://www.r-project.org/>
→ I have not updated R in a while; you'll probably get a 2026 reference

31

31

citation("psych")

To cite package 'psych' in publications use:

William Revelle (2024). *_psych: Procedures for Psychological, Psychometric, and Personality Research_*. Northwestern University, Evanston, Illinois. R package version 2.4.6, <<https://CRAN.R-project.org/package=psych>>.

A BibTeX entry for LaTeX users is

```
@Manual{,
  title = {psych: Procedures for Psychological, Psychometric, and
  Personality Research},
  author = {{William Revelle}},
  organization = {Northwestern University},
  address = {Evanston, Illinois},
  year = {2024},
  note = {R package version 2.4.6},
  url = {https://CRAN.R-project.org/package=psych},
}
```

APA style (I think correct...):

Revelle, W. (2024). *psych: Procedures for personality and psychological research* (Version 2.4.6) [Computer software]. <https://CRAN.R-project.org/package=psych>

32

32

Overview

- **Book chapters 1 and 2: important concepts**
 - Population vs. sample; descriptive vs. inference statistics
 - Types of data and probability distributions (discrete vs. continuous)
 - Central limit theorem; central tendency, variability, shape dispersion
- **More R basics: data management**
 - Recap/leftovers/add-on from last week
 - **Saving and loading data and your work**
 - **Long vs. wide format: reshaping**
 - **Selecting parts of a data frame (subset)**
- **Homework**

33

33

New Stuff

- Computing new variables and adding them to a data frame
- Saving your work (data, scripts, work spaces)
- Reshaping data from wide to long format and back
- More data management: subsetting (more next week!)

34

34

Computing/adding new variables to a data frame

- As example, let's do the steps necessary to compute the variance of a variable (see book chapter 1)
- Let's use the variable **Psycho** from the parenting data (see [ExampleScript_Week02_11Sept2026.R](#) on Brightspace)

```
library(foreign)
df_parenting <- read.spss('parenting.sav',
  to.data.frame = TRUE)
head(df_parenting)
tail(df_parenting)
summary(df_parenting) # no NAs in Psycho, good!
densityplot(df_parenting$Psycho)
```

35

(1) Subtract the mean of the variable from each value in the variable. Add this new variable to the data frame:

```
df_parenting$DiffMean_Psycho <-
df_parenting$Psycho - mean(df_parenting$Psycho)
```

Check what you created (e.g., to make sure it is not just all NA entries):

```
df_parenting$DiffMean_Psycho
or
head(df_parenting)
```

36

```
head(df_parenting)
```

	PARTNO	Gender	Response	Demand	Esteem	Control	Psycho	SES	Problems	Neurotic	DiffMean_Psycho
1	100	male	low	low	8	5	36	27	20	3	1.04878
2	110	female	high	high	8	5	36	58	14	5	1.04878
3	120	female	low	low	8	5	29	62	10	4	-5.95122
4	130	male	low	low	8	6	29	77	17	11	-5.95122
5	140	male	low	low	8	7	24	62	14	2	-10.95122
6	150	female	high	low	9	5	29	62	11	7	-5.95122

37

(2) Next, we have to square these values

```
df_parenting$SQ_DiffMean_Psycho <-  
df_parenting$DiffMean_Psycho ^ 2
```

Check what you did:

```
head(df_parenting)
```

or

```
df_parenting$SQ_DiffMean_Psycho
```

38

(3) Sum up these squared values to get the "**sum of squares**" (important quantity in statistics)

```
SumSquares_Psycho <-  
  sum(df_parenting$SQ_DiffMean_Psycho)  
SumSquares_Psycho # 16215.12
```

(4) To compute the **variance**, we divide the SS by the number of observations

(4.1) Number of observations:

```
n_Psycho <-  
  length(df_parenting$SQ_DiffMean_Psycho)  
n_Psycho # 369
```

39

(4.2) Divide the SS by the number of observations

```
VAR_Psycho <- SumSquares_Psycho / n_Psycho  
VAR_Psycho # 43.94342
```

If we compare this to what we get with var()

```
var(df_parenting$Psycho) # 44.06283
```

Why is the number from var() slightly bigger?

- Descriptive versus inferential variance
- Inferential variance estimates the **population** variance

```
VAR_Psycho_Inf <- SumSquares_Psycho / (n_Psycho - 1)  
# 44.06283
```

SD is the square root of the variance

```
sqrt(VAR_Psycho_Inf) # 6.637984  
sd(df_parenting$Psycho) # 6.637984
```

40

New Stuff

- Computing new variables and adding them to a data frame
- **Saving your work (data, scripts, work spaces)**
- Reshaping data from wide to long format and back
- More data management: subsetting (more next week!)

41

41

How to best save your work

- Syntax files (extension .R)
- Data frames (recommended: .csv)
- Work spaces (extension .RData)
- R objects (extension .rds)

42

How to best save your work

- **Syntax files (extension .R)**
- Data frames (recommended: .csv)
- Work spaces (extension .RData)
- R objects (extension .rds)

43

Syntax Files

- **Thou shalt not** type all your commands in the Console window
 - **Thou shalt** type them into a syntax file; for they are easy to save (and you can take them home and continue on your computer at home, for example)
- An R syntax file is like an SPSS syntax file
(basically just a text file containing commands and comments)

44

Syntax Files cont.

- **Create a new empty syntax file** (just as a new empty word document) in the R menu
File → New → R Script
- Save and open syntax files using the menu items:
 - File → Save as... (or File → Save)
 - File → Open File... (or double click the file you want to open)

45

How to best save your work

- Syntax files (extension .R)
- **Data frames (recommended: .csv)**
- Work spaces (extension .RData)
- R objects (extension .rds)

46

Data Frames

The main object type for your data

- **Columns:** Variables (pp_code, f_gender, age, ...)
 - **My recommendation:**
 - Use variable names like **f_...** (for factors), **o_...** (for ordered factors), **n_...** (for numerical), etc
 - Your future self will thank you
- **Rows:** Data units; e.g.
 - Participants (e.g., SPSS) → “**wide format**”
 - Repeated measures (trials) of a within-subject design
→ “**long format**” **[this week!]**

47

47

Data Frames

Indexing: MyThesisData1[**rows**, **columns**]

- MyThesisData1[1, 1]
- MyThesisData1[, 3]
- MyThesisData1[2:5, c(1,3)]
- MyThesisData1[2:5, c("pp_code", "f_gender")]

48

48

Some recommendations

- If it's a categorical variable, use a factor with entries like "brown-eyed" and "blue-eyed" etc, not numbers
- If it's a numeric variable, use a numeric variable without value levels
- Safer to avoid any potential mixup (do not treat a factor like a continuous variable and vice versa)
- You'll learn how you can turn, e.g., entries like 1 and 2 into entries like "brown-eyed" and "blue-eyed" (= **recoding**)

49

Data frames

- We **import** a file (data frame) using, e.g.:
 - `data_1 <- read.csv("C:/MyFolder/MyRawData.csv")`
- then, we might do something with it (like transformations, creating new variables, etc), leading to a new data frame called `data_2`
- To **save** this new data frame, use something like:
 - `write.csv(data_2, file = "C:/MyFolder/MyPreprocessedData.csv", row.names = FALSE)`

50

Data frames cont.

- Next time, you **load the new data** frame using:
- `data_2 <-
 read.csv("C:/MyFolder/MyPreprocessedData.csv")`

If you have specified the working directory beforehand:

```
write.csv(data_2, file = "MyPreprocessedData.csv",  
row.names = FALSE)  
data_2 <- read.csv("MyPreprocessedData.csv")
```

51

How to best save your work

- Syntax files (extension .R)
- Data frames (recommended: .csv)
- **Work spaces (extension .RData)**
- R objects (extension .rds)

52

Work Spaces

- The work space is like a snapshot of your current R environment
- It contains all your **objects**, i.e., variables, data frames, and results of your analyses; *as long as you did something like:*
`model_1 <- lm(...)`
- Sometimes, models/commands run for a looooong time
- In these cases, it can be particularly valuable if you don't have to rerun them again!

53

Work Spaces cont.

- It is then a good idea to save the workspace **in addition to** saving data frames and syntax files
 - `save.image("C:/MyFolder/Workspace_HW2.RData")`
 - **Next time, you can then load that work space:**
 - `load("C:/MyFolder/Workspace_HW2.RData")`
- Everything that you have put into an object (like `model_1`), will be there again!
- But you need to again load the packages, set your options (e.g., `scipen`), contrast settings, etc

54

How to best save your work

- Syntax files (extension .R)
- Data frames (recommended: .csv)
- Work spaces (extension .RData)
- **R objects (extension .rds):**
commands `saveRDS()`, `readRDS()`
→ save and load single objects from workspace (e.g., single data frame or vector)
<https://fromthebottomoftheheap.net/2012/04/01/saving-and-loading-r-objects/>

55

New Stuff

- Computing new variables and adding them to a data frame
- Saving your work (data, scripts, work spaces)
- **Reshaping data from wide to long format and back**
- More data management: subsetting (more next week!)

56

56

Wide vs. Long format

Example: Data frame

- **20 participants (10 f, 10 m)**
- **Rated their mood 3 times**
(0 = not positive at all; 100 = very positive)
 - 1st: baseline
 - 2nd: after watching a short funny video clip
 - 3rd: after watching a short sad video clip

Wide format

- Each rating is represented in a column
(familiar, e.g., from SPSS)
- Brightspace Week 2: Rating_ExampleData.csv

57

57

```
r_0 <- read.csv("Rating_ExampleData.csv")
head(r_0)
  pp_code f_gender rating_1 rating_2 rating_3
1  pp_1   female      61      82      32
2  pp_2   female      14      36       0
3  pp_3   female      58      80      28
4  pp_4   female      25      42       0
5  pp_5   female      39      62       2
6  pp_6   female      61      81      30

tail(r_0)
  pp_code f_gender rating_1 rating_2 rating_3
15 pp_15   male      58      80      25
16 pp_16   male      76      94      48
17 pp_17   male      30      53       0
18 pp_18   male      49      66      23
19 pp_19   male      50      68      22
20 pp_20   male      60      77      30
```

58

58

Wide vs. Long format

Long format (“stacked”)

- (Repeated) measures are not represented in different columns, but all in the same column
- A separate variable indicates whether the data belong to the first, second, third, etc repeated measure

Thus, in our example

- Each participant is represented as 3 rows
 - Rating 1
 - Rating 2
 - Rating 3

59

59

Wide

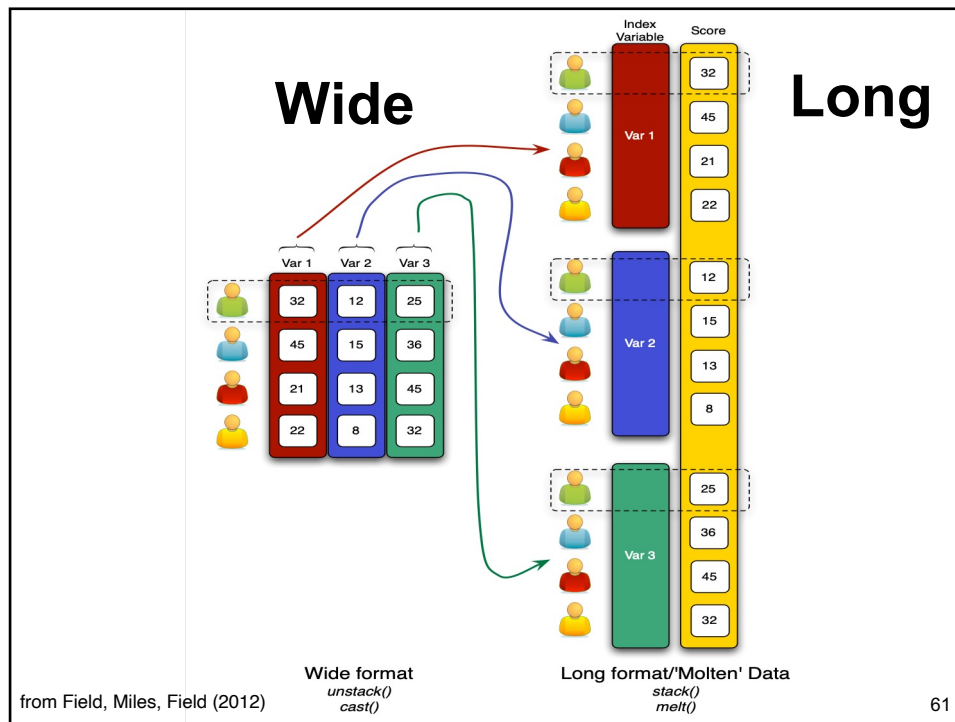
	Var 1	Var 2	Var 3
	32	12	25
	45	15	36
	21	13	45
	22	8	32

Wide format
`unstack()`
`cast()`

from Field, Miles, Field (2012)

60

60



61

61

Long format ("stacked")

- Most analyses in R use long format
- When you program a computer task for your experiment, output files are often in long format
 - Each trial is represented in a new row in the data file
- Very important to know how to change from wide to long format, and back ("reshaping")

62

62

Reshaping Data

Go from long to wide format and vice versa

- `stack()`, `unstack()`
- `melt()`, `cast()` from library `reshape`
- `reshape()` (comes with basic R installation)

`stack()`, `unstack()`

→ Only for very simple data; not recommended, really

(I give a short example in this week's R example script
ExampleScript_Week02_11Sept2026.R)

63

63

```
r_0
  pp_code f_gender rating_1 rating_2 rating_3
1    pp_1   female      61      82      32
2    pp_2   female      14      36       0
3    pp_3   female      58      80      28
4    pp_4   female      25      42       0
5    pp_5   female      39      62       2
6    pp_6   female      61      81      30
7    pp_7   female      52      79      25
8    pp_8   female      27      42       0
9    pp_9   female      23      39       0
10   pp_10  female      52      72      19
11   pp_11   male      30      47       0
12   pp_12   male      24      39       0
13   pp_13   male      40      57       6
14   pp_14   male      57      71      30
15   pp_15   male      58      80      25
16   pp_16   male      76      94      48
17   pp_17   male      30      53       0
18   pp_18   male      49      66      23
19   pp_19   male      50      68      22
20   pp_20   male      60      77      30
```

64

64

melt(), cast()

```
data2 <- melt(data1, id = c(constant variables),
              measured = c(variables changing among columns))
```

for example with repeated measures

```
data2 <- melt(data1, id = c("pp_code", "gender", "age"),
              measured = c("trial1", "trial2", "trial3", "trial4"))
```

for our rating data

- what are the id variables?
- what are the measured variables?

	pp_code	f_gender	rating_1	rating_2	rating_3
1	pp_1	female	61	82	32
2	pp_2	female	14	36	0
3	pp_3	female	58	80	28

65

65

```
r_1_melt <- melt(r_1, id = c("pp_code", "f_gender"),
                 measured = c("rating_1", "rating_2", "rating_3"))
```

	pp_code	f_gender	variable	value
1	pp_1	female	rating_1	61
2	pp_2	female	rating_1	14
3	pp_3	female	rating_1	58
4	pp_4	female	rating_1	25
5	pp_5	female	rating_1	39
6	pp_6	female	rating_1	61
7	pp_7	female	rating_1	52
8	pp_8	female	rating_1	27
9	pp_9	female	rating_1	23
10	pp_10	female	rating_1	52
11	pp_11	male	rating_1	30
12	pp_12	male	rating_1	24
13	pp_13	male	rating_1	40
14	pp_14	male	rating_1	57
15	pp_15	male	rating_1	58

66

66

	pp_code	f_gender	variable	value
1	pp_1	female	rating_1	61
2	pp_2	female	rating_1	14
3	pp_3	female	rating_1	58
4	pp_4	female	rating_1	25
5	pp_5	female	rating_1	39
6	pp_6	female	rating_1	61
7	pp_7	female	rating_1	52
8	pp_8	female	rating_1	27
9	pp_9	female	rating_1	23
10	pp_10	female	rating_1	52
11	pp_11	male	rating_1	30
12	pp_12	male	rating_1	24
13	pp_13	male	rating_1	40
14	pp_14	male	rating_1	57
15	pp_15	male	rating_1	58
16	pp_16	male	rating_1	76
17	pp_17	male	rating_1	30
18	pp_18	male	rating_1	49
19	pp_19	male	rating_1	50
20	pp_20	male	rating_1	60
21	pp_1	female	rating_2	82
22	pp_2	female	rating_2	36
23	pp_3	female	rating_2	80
24	pp_4	female	rating_2	42
25	pp_5	female	rating_2	62
26	pp_6	female	rating_2	81

67

cast()

from long to wide format

```
data_wide <- cast(data_long, id_variables ~
  measured_variable)
```

Example

- ID variables: pp_code, gender, age
- repeated-measures variable: response_time

```
data3 <- cast(data2, pp_code + gender + age ~
  response_time)
```

in simple cases, this also works

```
data3 <- cast(data2)
```

BUT: there needs to be exactly one variable called `variable` and exactly one variable called `value`

68

68

```
r_1_melt <- melt(r_1, id = c("pp_code", "f_gender"),
  measured = c("rating_1", "rating_2", "rating_3"))
```

	pp_code	f_gender	variable	value
1	pp_1	female	rating_1	61
2	pp_2	female	rating_1	14
3	pp_3	female	rating_1	58
4	pp_4	female	rating_1	25
5	pp_5	female	rating_1	39
6	pp_6	female	rating_1	61
7	pp_7	female	rating_1	52
8	pp_8	female	rating_1	27
9	pp_9	female	rating_1	23
10	pp_10	female	rating_1	52
11	pp_11	male	rating_1	30
12	pp_12	male	rating_1	24
13	pp_13	male	rating_1	40
14	pp_14	male	rating_1	57
15	pp_15	male	rating_1	58

69

69

	pp_code	f_gender	variable	value
1	pp_1	female	rating_1	61
2	pp_2	female	rating_1	14
3	pp_3	female	rating_1	58
4	pp_4	female	rating_1	25

Therefore, this works in this specific case

```
r_1_cast <- cast(r_1_melt)
```

This is the full command (which does the same)

```
r_1_cast_b <- cast(r_1_melt, pp_code + f_gender
  ~ variable)
```

70

70

If we had changed the variable names after melting

	pp_code	f_gender	rating_1or2or3	rating
1	pp_1	female	rating_1	61
2	pp_2	female	rating_1	14
3	pp_3	female	rating_1	58
4	pp_4	female	rating_1	25

This is the full command, where we also specify which variable is the "repeated-measures" variable

```
r_1_cast_b <- cast(r_1_melt, pp_code + f_gender
~ rating_1or2or3, value = "rating")
```

71

71

reshape()

```
data2 <- reshape(data1, idvar = "pp_code", varying =
c("rating_1", "rating_2", "rating_3"), timevar =
"rating_1or2or3", v.names = "rating", direction = "long")
```

- **idvar** variable indicating “who is who” (for example the participant code)
- **varying** the columns we want to stack
- **timevar** the variable that will indicate—in the new stacked data frame—from which original column the data point came
- **v.names** the name of the new stacked variable (in our example the ratings 1, 2, 3)
- **direction** whether the reshape is from wide to **long** or from long to **wide** (you specify how it should look *after* it has been reshaped, i.e., what you want, not what you have)
- for long → wide, see my R example script on Brightspace

72

72

New Stuff

- Computing new variables and adding them to a data frame
- Saving your work (data, scripts, work spaces)
- Reshaping data from wide to long format and back
- **More data management: subsetting (more next week!)**

73

73

Selecting parts of a data frame

We might want to create a smaller data frame, that consists only some of the observations (= rows) or only some of the variables (= columns), or both

- `subset()`
- `==`
- `!=`
- `droplevels()`
- `which()`

74

74

	pp_code	f_gender	variable	value
1	pp_1	female	rating_1	61
2	pp_2	female	rating_1	14
3	pp_3	female	rating_1	58
4	pp_4	female	rating_1	25
5	pp_5	female	rating_1	39
6	pp_6	female	rating_1	61
7	pp_7	female	rating_1	52
8	pp_8	female	rating_1	27
9	pp_9	female	rating_1	23
10	pp_10	female	rating_1	52
11	pp_11	male	rating_1	30
12	pp_12	male	rating_1	24
13	pp_13	male	rating_1	40
14	pp_14	male	rating_1	57
15	pp_15	male	rating_1	58
etc...				

75

75

subset()

```
df2 <- subset(df1, subset, select)
```

- `subset` specifies the observations (rows) to keep
- `select` specifies the variables (columns) to keep

Let's say we want only the data for females

```
r_1_melt_f <- subset(r_1_melt, subset = f_gender == "female")
```

```
r_1_melt_f <- subset(r_1_melt, f_gender == "female")
```

What is this doing?

```
subset(r_1_melt, f_gender != "female")
```

76

76

Let's say we want only the variables `pp_code` and `gender` (but keep all observations, i.e., all rows)

You already learned a few ways how to do this

```
r_1_melt_ppg <- r_1_melt[, 1:2]
r_1_melt_ppg <- r_1_melt[, c("pp_code",
"f_gender")]
```

With `subset()`

```
r_1_melt_ppg <- subset(r_1_melt, select =
c(pp_code, f_gender))
r_1_melt_ppg <- subset(r_1_melt, select =
c("pp_code", "f_gender"))
```

77

77

Combine selecting rows and columns

```
r_1_melt_small <- subset(r_1_melt, subset =
variable != "rating_2", select = -f_gender)
```

This works as well (without `subset =`)

```
r_1_melt_small <- subset(r_1_melt, variable !=
"rating_2", select = -f_gender)
```

Notes

- `- "f_gender"` doesn't work!
- `"rating_2"` has to be in quotation marks (it is a specific value in that variable)
- For `"rating_2"`, you need to use `!=` as `- "rating_2"` doesn't work

78

78

Combine several criteria with logical "and" & and logical "or" |

- Data frame diamonds from package ggplot2
- It's not a data frame but a tibble → package tibble
<https://cran.r-project.org/web/packages/tibble/vignettes/tibble.html>

```
diamonds_df <- as.data.frame(diamonds)
```

```
head(diamonds_df)
```

carat	cut	color	clarity	depth	table	price	x	y	z
0.23	Ideal	E	SI2	61.5	55	326	3.95	3.98	2.43
0.21	Premium	E	SI1	59.8	61	326	3.89	3.84	2.31
0.23	Good	E	VS1	56.9	65	327	4.05	4.07	2.31
0.29	Premium	I	VS2	62.4	58	334	4.20	4.23	2.63
0.31	Good	J	SI2	63.3	58	335	4.34	4.35	2.75
0.24	Very Good	J	VVS2	62.8	57	336	3.94	3.96	2.48

79

79

Select rows with Ideal, Very Good, and Good cuts

```
diamonds_df_2 <- subset(diamonds_df, cut ==  
"Ideal" | cut == "Very Good" | cut == "Good")
```

Another option

```
diamonds_df_2 <- diamonds_df[diamonds_df$cut  
%in% c("Ideal", "Very Good", "Good"),]
```

80

80

droplevels()

This command removes all unused factor levels

Best to **ALWAYS** use it when you select parts of a data frame with commands like `subset()`, `which()`, etc!

```
r_1_melt_f <- subset(r_1_melt, f_gender == "female")
r_1_melt_f$f_gender
```

```
[1] female female female female female female female female female
[10] female female female female female female female female female
[19] female female female female female female female female female
[28] female female female
```

Levels: female **male**

```
r_1_melt_f <- droplevels(subset(r_1_melt, f_gender == "female"))
```

```
[1] female female female female female female female female female ...
Levels: female
```

I am serious, ALWAYS use it when you use a command like `subset()`, `which()`, etc!

81

81

Overview

- **Book chapters 1 and 2: important concepts**
 - Population vs. sample; descriptive vs. inference statistics
 - Types of data and probability distributions (discrete vs. continuous)
 - Central limit theorem; central tendency, variability, shape dispersion
- **More R basics: data management**
 - Recap/leftovers/add-on from last week
 - Saving and loading data and your work
 - Long vs. wide format: reshaping
 - Selecting parts of a data frame (`subset`)
- **Homework**

82

82

Homework/Assignments

Detailed instructions in

"Homework_AdvStatsR_Week2.docx" on Brightspace

1. Work through my class example script
ExampleScript_Week02_11Sept2026.R
 2. Do some basic descriptive statistics with the parenting data frame
 3. Create your own data frame, do some reshaping, and create some figures
- Put the required answers, figures, and R code in a word doc, create a pdf of it, and upload it to the Pitch2Peer portal before 10:20 next Monday.

83

83

Lab session

Peergrading of homework 1

- Approx. first 15 min of each lab session
- **Everybody** needs to peer-grade a homework from another student (will be randomly assigned to you) [deadline is Sept 21 at 10:20]
- **Grading key for peergrading:** Brightspace → Week 1 → Homework → [BS86_HW1_2026_AnswersForGrading.pdf](#)

Very detailed homework example solution R script

not for grading, but if you want to compare your own solution in detail to mine: Brightspace → Week 1 → Homework → [HW_Week01_ExampleSolution_11Sept2026.R](#)

84

84

Tips/instructions for peer-grading

see first page of [BS86_HW1_2026_AnswersForGrading.pdf](#)

- We always check 10 things → max 10 points per HW
- For each of the 10 things, you can give 0 to 1 point
- Main goal of peer-grading: making sure everybody does their homework (→ do not punish mistakes much; punish laziness)
- Point out where and why the student did not obtain the full point;
try to be informative and constructive
- Indicate the total number of points for the homework (5.5 points or more: pass; fewer than 5.5 points: fail)
- Some examples on first page of [BS86_HW1_2026_AnswersForGrading.pdf](#)
- **Deadline to finish peer-grading: Monday Sept 21 at 10:20!**

85

85

Questions or comments?

See you after lunch!

86

86